COMPLETE PROJECT DOCUMENTATION

AgriSense

AI-Powered Smart Agriculture Drone
& Soil Health Station

See from above • Sense the soil • Water only where needed

PROJECT TYPE	Embedded + IoT + AI hardware project
HARDWARE	Arduino UNO | ESP32 | Raspberry Pi 4 | F450 quadcopter
AI MODEL	MobileNetV2 transfer learning, 15 plant-disease classes
TEAM	Team Micro Maniacs (Group 1) Kazi Shibli Zaman Rahil (main developer) Hasan Al Banna | Syed Rafid Sumaita Binte Hafiz | Nazifa Tabassum
INSTITUTION	Dept. of CSE, United International University (UIU)
DOCUMENT	Version 1.0 - 10 November 2026

Prepared from the complete project source code, database scripts and files.

Table of Contents

1. Executive Summary . . . 3
What the team built . . . 3
2. Problem Statement and Objectives . . . 4
3. Scope: What Is in This Repository . . . 4
4. Team and Contributions . . . 5
5. System Architecture . . . 5
5.1 Decision flow implemented in the firmware . . . 6
6. Hardware . . . 6
6.1 Components and cost . . . 6
6.2 Ground-station pin map (from the firmware) . . . 7
6.3 Sensor and actuator roles . . . 8
7. Ground-Station Firmware . . . 8
7.1 Configuration constants . . . 8
7.2 Moisture calibration . . . 8
7.3 Zone routing with two servos . . . 9
7.4 Observations on the current firmware . . . 9
8. AI Plant-Health Detection . . . 9
8.1 Dataset . . . 9
8.2 Preprocessing . . . 10
8.3 Model . . . 10
8.4 detection.py step by step . . . 11
8.5 capture.py . . . 11
9. Web Dashboard (Supporting Subsystem) . . . 11
10. Project Timeline . . . 12
11. Test Plan . . . 12
12. Safety and Reliability Notes . . . 13
13. Limitations and Future Work . . . 13
13.1 Limitations . . . 13
13.2 Future work . . . 14
14. Repository Guide and How to Run . . . 14
Run the ground station . . . 14
Run the vision scripts . . . 14
15. Presentation Guide and Viva Questions . . . 15
15.1 Suggested 8-minute demo . . . 15
15.2 Likely questions and model answers . . . 15
Appendix - Glossary . . . 16

1. Executive Summary

AgriSense is a two-tier precision-agriculture prototype built by **Team Micro Maniacs** (Group 1, Department of Computer Science and Engineering, United International University). A **quadcopter** carrying a Raspberry Pi and camera surveys crops and uses a deep-learning model to spot plant disease, while a **ground station** built around an Arduino UNO and an ESP32 measures soil moisture, air temperature and humidity and decides **where and when to water**. A servo-actuated manifold sends water from one pump to several zones, so each zone gets only what it needs.

The idea is a feedback loop: *the drone sees the big picture, the ground station handles the fine details*. The report's decision rule is simple: **IF** soil moisture is below the threshold **AND** the UAV reports stress in that zone **THEN** water that zone.

2	15	20,639	92.09%	5 weeks
Tiers: UAV + ground	Disease classes	Training images	Validation accuracy (reported)	Concept to demo



Figure 1 - The quadcopter (UAV tier).



Figure 2 - Ground station set-up for soil monitoring.

What the team built

- **A flying drone** - F450 frame, F405 V3 PLUS flight controller, four ESCs, DJI 2212 motors, 3S LiPo; passed its first controlled flight test.
- **A ground station** - three soil-moisture probes, a DHT22, a 4-channel relay, a pump and a two-servo irrigation manifold.
- **An AI plant-health model** - MobileNetV2 fine-tuned on a 15-class PlantVillage subset, with background removal (Rembg) in preprocessing.
- **A web dashboard** - live readings, camera preview and capture, AI analysis and irrigation status (shown in the report).
- **Software in this repository** - the Arduino firmware, the Pi camera-capture script and the disease-detection script.

2. Problem Statement and Objectives

Farming still applies water, fertiliser and pesticide **uniformly**, regardless of what each area needs, and crop problems are often noticed only when damage is visible. That wastes scarce water and delays treatment. AgriSense combines **aerial imaging** with **on-ground sensing** so that resources follow evidence instead of fixed schedules.

#	Objective (from the project report)	How the prototype addresses it
1	Vision-guided UAV platform that scans fields and maps problem areas	Quadcopter + Raspberry Pi + Camera V2; MobileNetV2 classifier (detection.py)
2	Ground station that tracks soil moisture, humidity and temperature	Arduino UNO with 3 soil probes and a DHT22 (GroundStation.ino)
3	Precision resource allocation	Threshold-based pump control and a servo manifold that routes water to dry zones
4	Validate performance (water saving, early detection, productivity)	Lab demonstration; model validation accuracy; field trials remain future work

3. Scope: What Is in This Repository

The report describes the complete system. The repository contains the parts that were committed as code. This table keeps the two clearly apart.

Component	Status	Where
Ground-station firmware (sensing, thresholds, pump, servo routing)	**In repository**	firmware/GroundStation/GroundStation.ino
Camera capture script (Picamera2, 1920 x 1080)	**In repository**	raspberry-pi/capture.py
Disease detection script (Rembg + MobileNetV2)	**In repository**	raspberry-pi/detection.py
Trained model (plant_disease_model.h5, 23.7 MB) and class_indices.json	Described in report	Not in the submitted archive; place next to detection.py
Model training notebook	Described in report	Not in the submitted archive (detection.py refers to a 03_ step)
ESP32 gateway, decision fusion and OLED display code	Described in report	Not in the submitted archive
Drone-to-ground Wi-Fi report (zone ID, coordinates, severity) and GPS mapping	Described in report	Not in the submitted archive
Web dashboard	Shown in report (Figure 4)	Not in the submitted archive
Flight-controller set-up and first flight	Hardware milestone	Completed in week 3 of the schedule
Two-way telemetry (433 / 900 MHz) and backup power	Future work	Section 13

> **Reading guide**
> Several statements in the report (for example the Raspberry Pi running OpenCV colour thresholding, or the ESP32 hosting the decision logic) describe the intended architecture. In the committed code the **Arduino UNO** makes the irrigation decision by itself and the Pi uses a **neural-network classifier**. This document describes the code as it is and marks everything else as *reported*.

4. Team and Contributions

AgriSense was developed by **Team Micro Maniacs (Group 1)** at the Department of Computer Science and Engineering, **United International University (UIU)**. **Kazi Shibli Zaman Rahil** is the project owner and main developer.

Team member	Role
Kazi Shibli Zaman Rahil (GitHub: Rahil03-git)	**Project owner and main developer**
Hasan Al Banna	Team member - [add responsibility]
Syed Rafid	Team member - [add responsibility]
Sumaita Binte Hafiz	Team member - [add responsibility]
Nazifa Tabassum	Team member - [add responsibility]

5. System Architecture

The system is a **multi-tiered distributed network** whose two tiers communicate wirelessly. The UAV tier does macro-scale data acquisition (crop-health imaging). The ground tier does micro-scale acquisition (soil conditions), makes decisions and acts (irrigation).

Green = code in this repository **Amber = described in the report only**

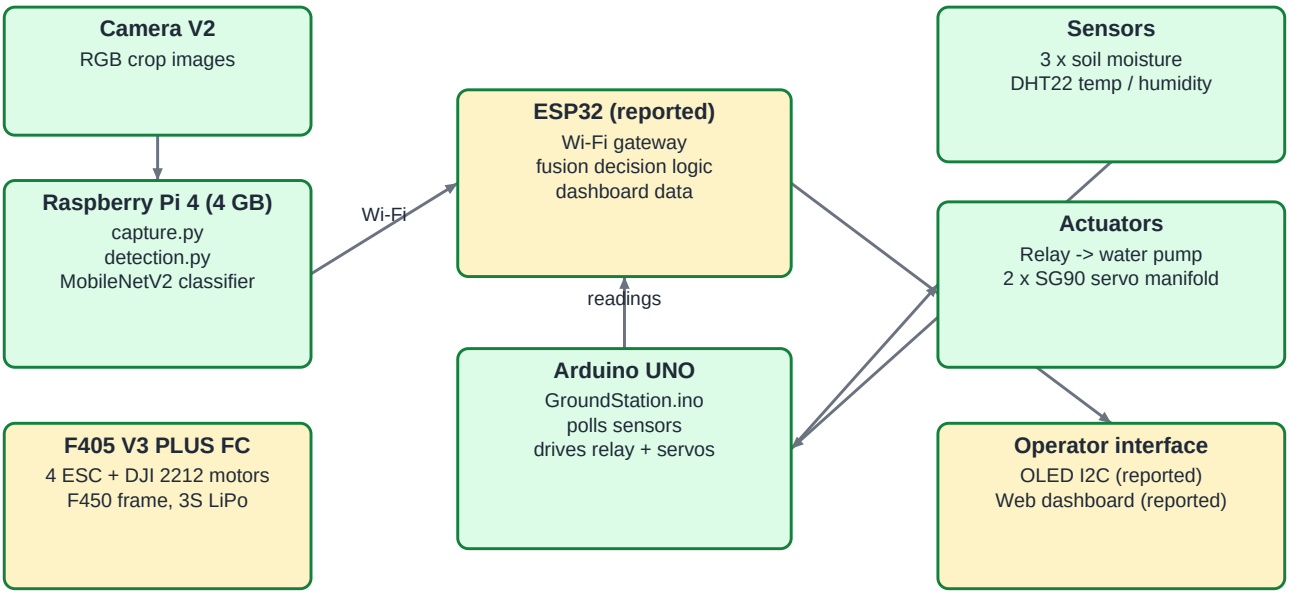


Figure 3 - Two-tier architecture. The flight controller works independently of the Raspberry Pi's vision workload.

Sub-system	Component	Role
Input (UAV)	Camera V2	Captures RGB images of the crop below

Sub-system	Component	Role
Data processing (UAV)	Raspberry Pi 4 (4 GB)	Onboard computer: runs the plant-disease classifier and prepares findings (zone, severity)
Flight control	F405 V3 PLUS FC, ESC, motors, propellers	Stability, motor speed and navigation, independent of the Pi
Communication	Raspberry Pi Wi-Fi to ESP32	Sends anomaly data (zone ID, coordinates, severity) to the ground station
Sensor aggregation	Arduino UNO	Stable analogue / digital polling of the low-power sensors
Decision logic	ESP32 (reported); Arduino UNO (in code)	Fuses UAV reports with soil readings and triggers watering
Actuation	4-channel relay, pump, servos	Relay switches the pump; servos route the water to the selected zone(s)
User interface	OLED I2C, web dashboard	Local status and remote monitoring and commands

5.1 Decision flow implemented in the firmware



Figure 4 - One pass of the main loop (repeats about every 2 seconds).

6. Hardware

6.1 Components and cost

Prices are the lowest retail prices the team found online in Bangladesh (BDT). The table lists the line items from the final report with the arithmetic recomputed.

Component	Sets	Unit (BDT)	Subtotal (BDT)
Automation and IoT (ground station)			
Arduino UNO (SMD CH340)	1	435	435
Jumper Wires (40 Pcs Set)	3	85	255
Breadboard (400 Tie Points)	3	87	261
4 Channel Relay (5V)	1	240	240
ESP-32 (WROOM32)	1	790	790
DHT-22 (Temp/Humidity Sensor)	1	235	235
Soil Sensor	3	150	450
Servo Motor (SG90)	2	180	360
12V 2A Charger/Adapter	1	120	120

Component	Sets	Unit (BDT)	Subtotal (BDT)
OLED Display I2C (0.96")	1	379	379
Water Pump (Mini DC 3-6V)	1	63	63
Subtotal ground station	**18**		**3,588**
Drone / FPV (UAV)			
Raspberry Pi 4GB	1	9,700	9,700
Camera V2	1	3,390	3,390
F405 V3 PLUS Flight Controller	1	8,499	8,499
ESC (Single Unit, e.g., 30A SimonK)	4	499	1,996
MATEK XT 60 PDB	1	948	948
DJI 2212 920KV VLDX Motor (Single Unit)	1	1,120	1,120
Propeller (1045 Pair)	2	140	280
F450 Quadcopter Frame	1	1,650	1,650
3500mAh Lipo Battery (3S)	1	3,500	3,500
High Temperature Silicone Wire (2ft Set)	2	130	260
20CM Strap Battery Binding (Single Strap)	2	47	94
Subtotal drone	**17**		**31,437**
TOTAL (sum of the lines above)	**35**		**35,025**

Bill-of-materials check

The report states **36 sets and 35,070 BDT**. The line items above add up to **35 sets and 35,025 BDT** - a difference of 1 set and 45 BDT. The motor line is also listed once (quantity 1) although a quadcopter uses **four** motors. Please confirm the final quantities before quoting a budget.

6.2 Ground-station pin map (from the firmware)

Function	Arduino UNO pin	Notes
DHT22 data	D2	Library DHT, type DHT22
Soil moisture - Zone 1	A0	Analogue input
Soil moisture - Zone 2	A3	Analogue input
Soil moisture - Zone 3 (main line / area)	A5	Analogue input
Servo 1 (Zone 1 gate)	D9	Closed = 180, open = 90
Servo 2 (Zone 2 gate)	D10	Closed = 0, open = 90
Pump relay	D4	Active-LOW module: HIGH = pump OFF, LOW = pump ON

6.3 Sensor and actuator roles

Part	Measures / does	Used for
Soil moisture probe (x3)	Volumetric water content (analogue)	Primary irrigation trigger per zone
DHT22	Air temperature and humidity	Environment readout; useful for evapotranspiration-based tuning (printed only in the current code)
Camera V2	RGB images	Crop-health analysis on the Raspberry Pi
4-channel relay (5 V)	Low-power switch for the 12 V supply	Safely switches the pump from a digital pin
Mini DC pump	Moves water	Delivers water to the selected zone through tubing or a drip emitter
SG90 servos (x2)	Rotary positions	Act as a manifold / diverter to choose the irrigation zone
OLED I2C 0.96"	Text status	Local feedback such as "Moisture LOW" (reported)

7. Ground-Station Firmware

`GroundStation.ino` is a single Arduino sketch of about 190 lines. It needs the **DHT sensor library** (with Adafruit Unified Sensor) and the built-in `Servo` library, and prints a status report to the serial monitor at 9600 baud.

7.1 Configuration constants

Constant	Value	Meaning
`DRY_VALUE`	800	Raw analogue reading of bone-dry soil (high value)
`WET_VALUE`	300	Raw reading of fully wet soil (low value)
`IRRIGATION_THRESHOLD_PERCENT`	30	A zone needs water when moisture is below 30 %
`Z1_GATE_OPEN / CLOSED`	90 / 180	Servo 1 angles
`Z2_GATE_OPEN / CLOSED`	90 / 0	Servo 2 angles
`Loop delay`	2000 ms	Pause between readings; +1000 ms servo pauses and +1500 ms hold while watering

7.2 Moisture calibration

The raw reading is converted to a percentage with the Arduino `map()` function and clamped to 0-100:

```
moisturePercent = map(raw, DRY_VALUE, WET_VALUE, 0, 100);   // 800 -> 0 %, 300 -> 100 %
return constrain(moisturePercent, 0, 100);
```

Raw reading	800	700	651	650	550	400	300
Moisture %	0	20	29	30	50	80	100
Zone needs water?	Yes	Yes	Yes	No	No	No	No

Because `map()` uses integer arithmetic, 651 gives 29 % (below the threshold) while 650 gives exactly 30 % (not below). A zone is therefore treated as dry for raw readings above 650. Re-calibrate DRY and WET for your own probes and soil.

7.3 Zone routing with two servos

Three zones are served by **one pump and two servos**. The pump runs if *any* zone is dry. The servo angles depend on which zones are dry (Zone 3 is the main line):

Dry zones	Servo 1 (D9)	Servo 2 (D10)	Log message
none	180 (closed)	0 (closed)	Pump OFF, all gates closed
Zone 1 only	90	(unchanged)	Zone 1 gate OPENED
Zone 2 only	(unchanged)	90	Zone 2 gate OPENED
Zone 3 only	180	0	Zone 3 gate OPENED (both at closed positions)
Zones 1 + 2	0	90	Zone 1 and 2 gates OPENED
Zones 1 + 3	0	0	Zone 1 and 3 gates OPENED
Zones 2 + 3	180	180	Zone 2 and 3 gates OPENED
Zones 1 + 2 + 3	0	180	Zone 1, 2 and 3 gates OPENED

The meaning of each angle combination depends on how the diverter is built; the table shows what the code writes. In every dry case the relay is then driven LOW (pump ON) and the loop waits 1.5 s so the soil can absorb water.

7.4 Observations on the current firmware

- **Comment mismatch:** a comment says "moisture < 40 %" while the constant is 30.
- **No hysteresis:** the pump switches off as soon as every zone is at or above 30 %, so values hovering around the threshold can make it chatter. A higher stop threshold (e.g. start at 30 %, stop at 45 %) would avoid this.
- **Stale servo positions:** in the single-zone cases only one servo is written, and both servos are reset only when all zones are wet. Moving directly from "Zone 2 only" to "Zone 1 only" leaves Servo 2 at 90.
- **No watering time limit:** a failed sensor reading that stays dry would keep the pump running; a maximum run-time is advisable.
- **DHT22 values are only printed**, not used in the decision.
- **No drone input:** the sketch reacts to soil moisture only. The "UAV reports stress" condition from the report is not in this code.

8. AI Plant-Health Detection

8.1 Dataset

The model uses the **PlantVillage** dataset (Kaggle: *abdallahalidev/plantvillage-dataset*), a benchmark of more than 54,000 leaf images in 38 categories. The team selected a subset of **20,639 images for 15 classes** covering pepper, potato and tomato, balancing healthy and diseased examples across major crops.

#	Class	#	Class
1	Pepper bell - Bacterial spot	8	Tomato - Leaf mold
2	Pepper bell - Healthy	9	Tomato - Septoria leaf spot
3	Potato - Early blight	10	Tomato - Spider mites (two-spotted)
4	Potato - Late blight	11	Tomato - Target spot
5	Potato - Healthy	12	Tomato - Yellow leaf curl virus
6	Tomato - Bacterial spot	13	Tomato - Mosaic virus
7	Tomato - Early blight	14	Tomato - Healthy
		15	Not named in the report (see note)

Note: the report says 15 classes but names 14 of them. The standard 15-class Pepper / Potato / Tomato subset also includes *Tomato Late blight*, which is the likely missing entry - confirm with `class_indices.json`.

8.2 Preprocessing

All dataset and test images were passed through the open-source **Rembg** background remover and placed on a plain white background to reduce noise and make the model focus on leaf features. The same step is applied at prediction time in `detection.py`.

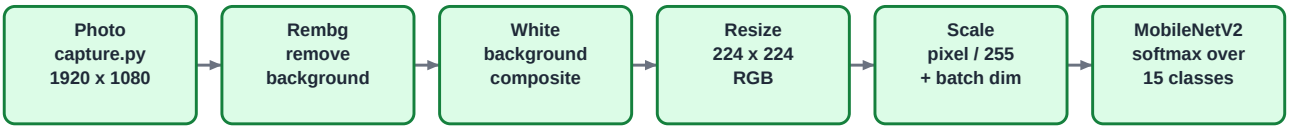


Figure 5 - Inference pipeline in detection.py.

8.3 Model

Property	Value (from the report)
Architecture	Convolutional neural network - MobileNetV2
Approach	Transfer learning, fine-tuned (the report also says "from scratch"; the details above indicate fine-tuning)
Parameters / size	About 2.5 million parameters; 23.7 MB (Keras .h5 file)
Input	224 x 224 RGB, scaled to 0-1
Classes / images	15 classes; 20,639 images
Validation accuracy	92.09 % (reported)
Training	30 epochs shown in the training plot

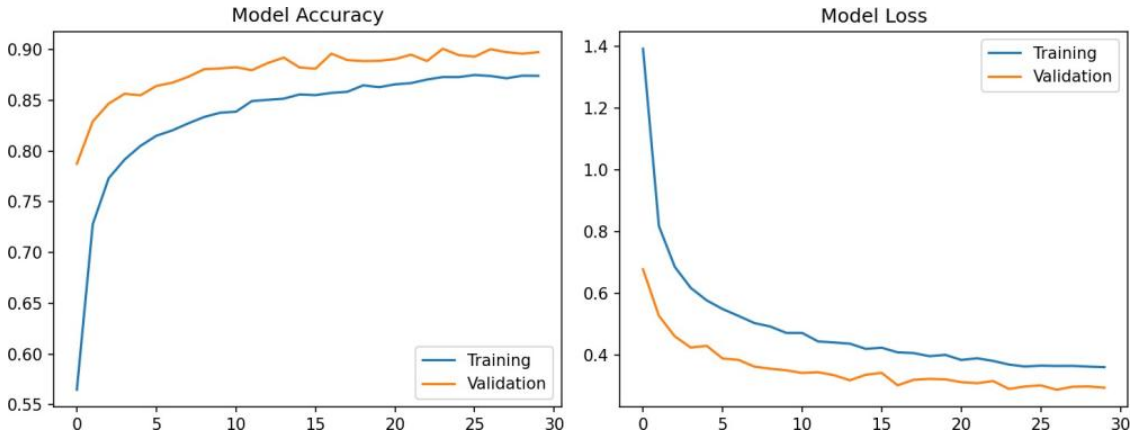


Figure 6 - Training and validation accuracy and loss over 30 epochs (from the report).

> **Accuracy figure**
> The text reports a validation accuracy of **92.09 %**, while the plotted validation curve levels off at about **0.90**. The two can differ if the figure comes from a different run or the number from a final evaluation on a held-out set. Re-run an evaluation on a test split and quote that number, with the confusion matrix, in future reports.

8.4 detection.py step by step

1. Load `plant_disease_model.h5` (`compile=False`) and `class_indices.json`; invert the mapping from class to index into index to class.
2. Open the image as RGBA, remove the background with Rembg, composite onto white, resize to 224 x 224 and convert to RGB.
3. Divide by 255, add a batch dimension and call `model.predict`.
4. Take the arg-max class and its probability and print the predicted disease with a confidence percentage.

The script contains a Colab path (`/content/1573847-WEB.jpg`) as its example image - change `example_image_path` before running locally.

8.5 capture.py

Uses **Picamera2**: starts a 640 x 480 preview, waits for the operator to press *Enter*, reconfigures for a still 1920 x 1080 capture and saves `image_YYYYMMDD_HHMMSS.jpg`. Ctrl+C cancels, and the camera is always stopped in the cleanup block.

9. Web Dashboard (Supporting Subsystem)

The dashboard is presented in the report as a supporting subsystem that illustrates usability and data fusion. It aggregates sensor data, shows the drone camera feed, displays scheduled and executed irrigation and allows remote commands. Its screens show three-zone soil moisture, air temperature, humidity, pump status and watering zone, plus *Start Preview* and *Image Capture* buttons and an **AI-Powered Plant Health Analysis** panel (plant identified, health status, detected disease). The dashboard code is not in the submitted archive.

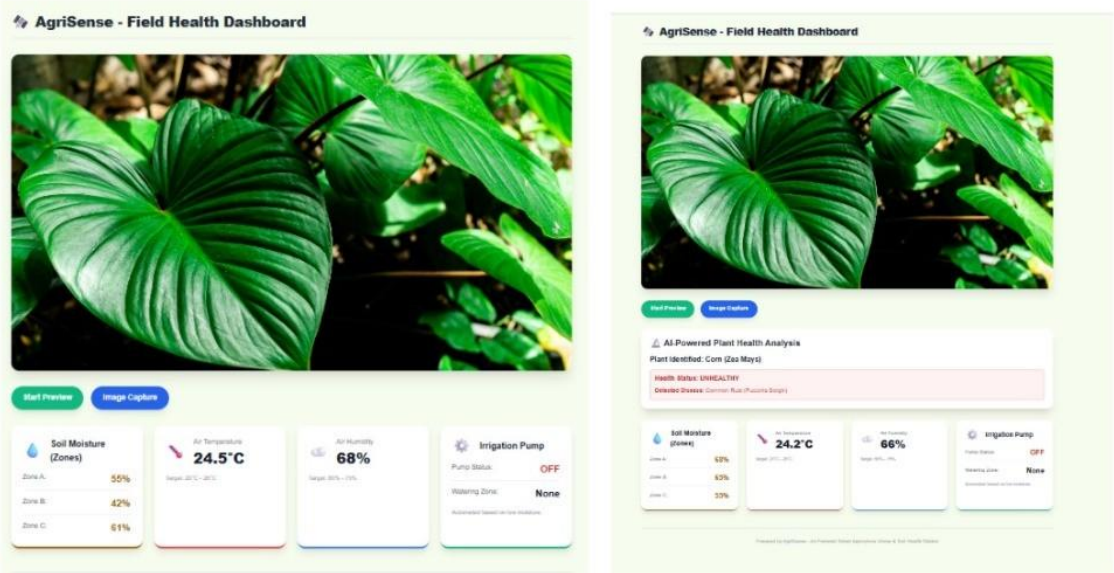


Figure 7 - AgriSense Field Health Dashboard (from the report).

10. Project Timeline

Week	Focus	Outcome
1	Planning and procurement	Full concept defined; 36 components procured
2	Ground station and hardware integration	Arduino / ESP32 connected to relay, DHT22, soil sensors and pump; firmware tests read data and trigger irrigation
3	Core flight system and first flight	F450 frame, motors, ESCs and F405 flight controller built and tuned; first controlled flight passed
4	Intelligence and vision integration	Raspberry Pi and Camera V2 mounted; software configured; model trained and optimised
5	Finalisation and system integration	Flight hardware merged with the AI system; wiring, battery mounts and placement refined; full-scale tests and a successful demonstration

11. Test Plan

These tests prove the main behaviours. Expected results follow from the code and the report.

ID	Test	Procedure	Expected result
T1	Sensor calibration	Read each probe in dry air / soil and in water; set DRY_VALUE and WET_VALUE	0 % when dry, about 100 % when wet
T2	Threshold	Adjust soil so a zone reads 31 %, then 29 %	No watering at 31 %; watering at 29 %
T3	Zone routing	Make each combination of zones dry	Servo angles match the table in Section 7.3; pump ON
T4	All wet	Wet all zones	Pump OFF; both gates closed
T5	Relay polarity	Observe the relay with the pump disconnected	HIGH on D4 = OFF, LOW = ON (active-LOW module)
T6	DHT22 failure	Unplug the DHT22	"DHT Read Failed" printed; irrigation logic still runs

ID	Test	Procedure	Expected result
T7	Camera capture	Run capture.py and press Enter	image_*.jpg saved at 1920 x 1080
T8	Disease prediction	Run detection.py on a leaf photo with the model present	Class name and confidence printed
T9	Missing model	Run detection.py without the .h5 file	"Model or class indices not found" message
T10	End to end	Drone flags a dry stressed zone; ground station checks moisture	Pump irrigates only that zone (reported behaviour)

12. Safety and Reliability Notes

- **Mains and 12 V supply:** keep the 12 V adapter and relay wiring insulated and away from water; use a flyback diode across the pump motor.
- **Water and electronics:** keep the Arduino, breadboard and relay outside the wet zone; protect the soil probes' connectors.
- **LiPo battery:** charge on a fireproof surface, never leave it unattended, add a low-voltage alert and fit a backup supply for the Raspberry Pi and flight controller.
- **Drone operation:** remove propellers while bench-testing; fly only in permitted areas and follow the local aviation authority's rules on drones.
- **Credentials:** the repository contains no Wi-Fi passwords or API keys. Keep any future credentials (ESP32 Wi-Fi, dashboard) out of Git.

13. Limitations and Future Work

13.1 Limitations

- The model is trained on **PlantVillage laboratory images** of three crops; real field photos from a drone differ in lighting, scale and background, so accuracy will be lower than the validation figure.
- Rembg background removal is computationally heavy on a Raspberry Pi and adds delay per image.
- Only 15 classes are recognised; unknown diseases and other crops are not covered.
- Communication between drone and ground station is limited (reported); zone mapping from GPS coordinates is not in the submitted code.
- The drone relies on a single LiPo battery with no backup power for critical electronics (reported).
- The irrigation logic has no hysteresis, no maximum run-time and no drone input (Section 7.4).

13.2 Future work

Area	Improvement	Benefit
Communication	Two-way telemetry (433 / 900 MHz) with a mission-control hub on the Arduino / ESP32 and a control laptop	Real-time mission updates, in-flight reprogramming, continuous data link
Energy	Backup battery or super-capacitor for the Raspberry Pi and flight controller; low-power code; low-voltage alert	Safer flights, longer and more predictable endurance
Irrigation	Hysteresis, maximum run-time, use of DHT22 data (evapotranspiration), scheduling	Less chatter, safer pump control, better water use
AI	Field-collected dataset, more crops and classes, test-set evaluation, quantised model for the Pi	Higher real-world accuracy and speed
Software	Commit the ESP32 code, dashboard and training notebook; add unit tests for the routing logic	A fully reproducible project
Validation	Field trials that measure water saving, early detection time and yield	Evidence for the project's objectives

14. Repository Guide and How to Run

Path	Content
`firmware/GroundStation/GroundStation.ino`	Arduino ground-station sketch
`raspberry-pi/capture.py, detection.py`	Camera capture and disease detection
`requirements.txt`	Python packages for detection.py
`hardware/`	Bill of materials (CSV) and pin map
`docs/`	Final project report, this documentation, figures

Run the ground station

1. Install the Arduino IDE and the libraries *DHT sensor library* and *Adafruit Unified Sensor* (Library Manager).
2. Open `firmware/GroundStation/GroundStation.ino`, choose *Arduino Uno* and the correct port.
3. Wire the pins listed in Section 6.2, upload, and open the Serial Monitor at 9600 baud.
4. Calibrate DRY_VALUE and WET_VALUE for your sensors before relying on the percentages.

Run the vision scripts

1. On the Raspberry Pi (64-bit OS with a camera): `python3 raspberry-pi/capture.py` and press Enter to capture a photo.
2. Install the detection packages: `pip install -r requirements.txt`.
3. Copy `plant_disease_model.h5` and `class_indices.json` next to `detection.py` and set `example_image_path`.
4. Run `python3 raspberry-pi/detection.py` from that folder.

15. Presentation Guide and Viva Questions

15.1 Suggested 8-minute demo

1. **Problem (1 min)** - blind watering wastes water; problems are found late.
2. **Architecture (1 min)** - two tiers and the decision rule.
3. **Ground station (2 min)** - show sensors, dry a probe, watch the pump and servo routing, open the serial monitor.
4. **AI (2 min)** - capture or load a leaf photo, show background removal and the prediction with confidence.
5. **Drone and dashboard (1 min)** - the flight video and dashboard screenshot.
6. **Limits and future work (1 min)** - telemetry, backup power, field data.

15.2 Likely questions and model answers

Q. Why two tiers?

The drone covers large areas quickly but cannot measure soil water; ground sensors measure soil precisely but only at a point. Combining them tells us where the problem is and what the soil needs.

Q. How is soil moisture converted to a percentage?

The raw analogue value is mapped linearly from the dry calibration value (800) to the wet value (300) onto 0-100 % and clamped. Each probe must be calibrated.

Q. When does the pump turn on?

When any zone's moisture is below 30 %. The relay is active-LOW, so the pin is driven LOW to run the pump and HIGH to stop it.

Q. How do two servos serve three zones?

The code writes a different pair of angles for each combination of dry zones, so the diverter can send water to zone 1, zone 2, the main line or a mix. Section 7.3 lists all combinations.

Q. Why use a relay?

The pump needs 12 V and more current than a microcontroller pin can supply. The relay isolates the low-power logic from the pump supply.

Q. What is transfer learning and why MobileNetV2?

We start from a network already trained on a large image set and fine-tune it for plant diseases. MobileNetV2 is small (about 2.5 M parameters, 23.7 MB) and fast, which suits a Raspberry Pi.

Q. Why remove the image background?

PlantVillage images have uniform backgrounds. Removing the background and using plain white makes field photos look more like the training data and lets the model focus on the leaf.

Q. How accurate is the model?

92.09 % validation accuracy was reported on 15 classes (the plotted curve shows about 0.90). Real field accuracy will be lower because training images are laboratory photos.

Q. What does the dashboard do?

It shows soil moisture per zone, temperature, humidity, pump status, the camera preview and the AI health analysis, and allows remote commands (shown in the report).

Q. What happens if the DHT22 fails?

The code prints a read-failed message and continues; irrigation depends only on soil moisture.

Q. What would you improve first?

Add hysteresis and a maximum pump run-time, then commit the ESP32 and dashboard code and evaluate the model on a held-out test set.

Q. How much did it cost?

About 35,000 BDT (the report states 35,070 BDT; the listed items sum to 35,025 BDT), most of it the drone and Raspberry Pi.

Appendix - Glossary

Term	Meaning
UAV	Unmanned aerial vehicle - the quadcopter drone
Precision agriculture	Applying water and inputs only where and when needed, guided by data
ESC	Electronic speed controller driving a brushless motor
PDB	Power distribution board
Active-LOW relay	A relay module that switches ON when its input pin is driven LOW
Transfer learning	Re-using a network trained on one task as the starting point for another
MobileNetV2	A lightweight convolutional network designed for mobile and embedded devices
Rembg	Open-source tool that removes image backgrounds
PlantVillage	Public benchmark dataset of healthy and diseased plant leaves
Evapotranspiration	Water lost from soil and plants to the air; depends on temperature and humidity